



Thanks for buying SSSMapGenerator!

If you liked the asset, please do give it a review!
Hopefully good ones...

If you have any comments, questions, suggestions for program improvements,
suggestions for additional features, bug reports, etc., please contact me at

E-mail : sw2.26tks@gmail.com
Twitter : https://twitter.com/sowa_jp

I'm Japanese, so I can understand Japanese perfectly. However, if you write in English, please use simple English.

▼About SSSMapGenerator

"SSSMapGenerator" is an asset that generates maps in the "Slay the Spire" style.
With flexible customization, it allows creating maps like "Slay the Spire" or unique ones based on your ideas.

▼Notes on SSSMapGenerator

"TextMeshPro" and "InputSystem" are used, so please install them beforehand (required).

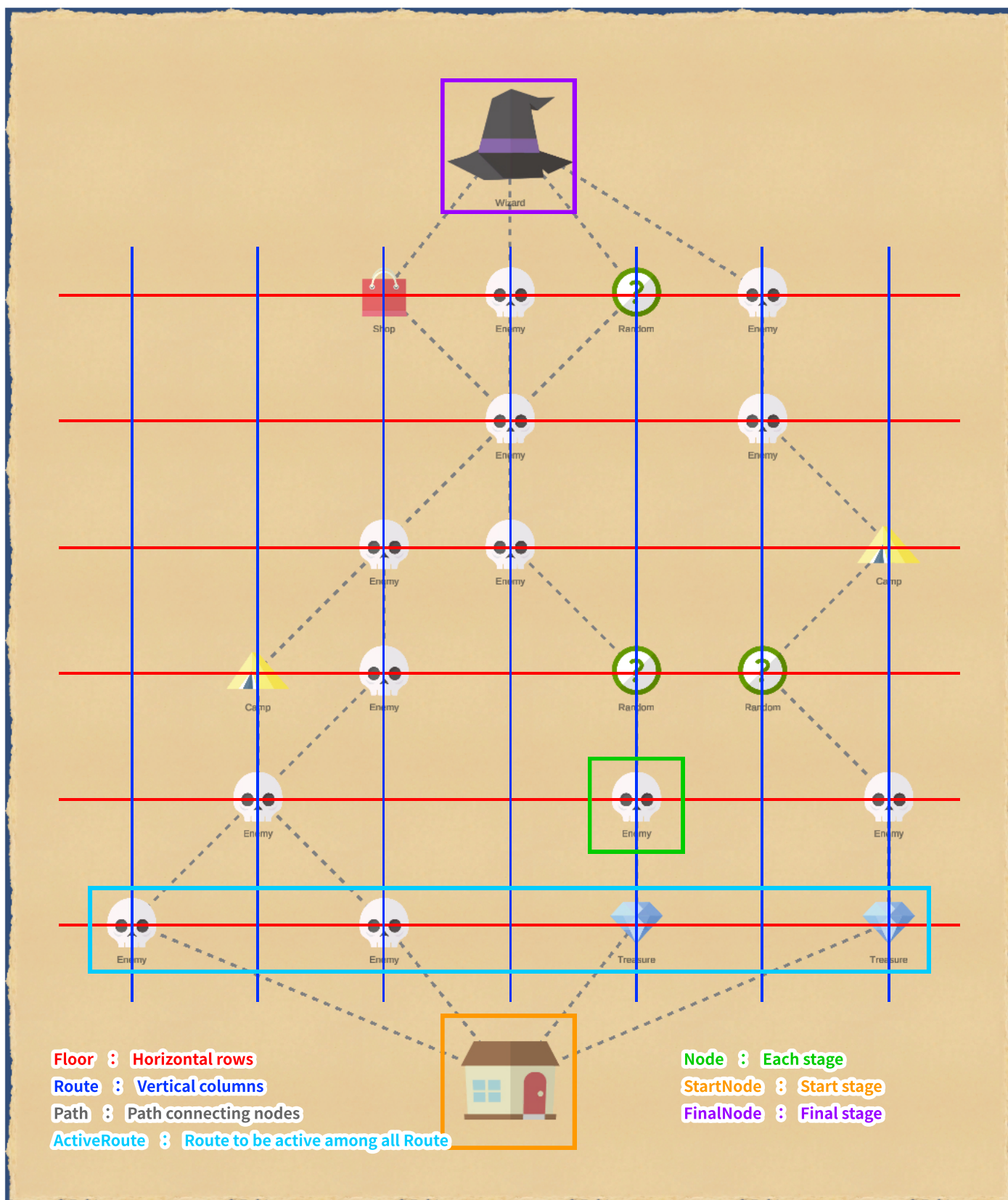
"SSSMapGenerator" does not have a map saving feature.

Since it uses a singleton, the map will be maintained during gameplay even when switching scenes,
but please note that you cannot resume from the middle of the map after the game ends.
However, using the seed allows for generating the same map at any time.

You will need to touch some programming to execute each node (enemy, shop, etc.).

Therefore, it is not recommended for those who do not understand C# at all or who create games solely with visual scripting.
The programming required is very simple (refer to the page on how to execute each node).

▼Basic

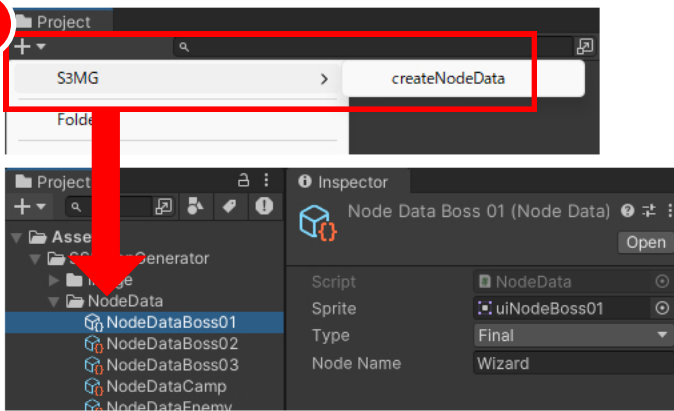


<Default Map Rules>

- Paths do not intersect: This can be changed in the settings.
- The same node type is not placed consecutively if possible: This can be changed in the settings.
However, depending on the settings, the rule may break, so at least one node type that can be placed consecutively must be registered.
If it still breaks, nodes will be assigned randomly.
- For nodes with branches, the destination must be a different type of node: This can be changed in the settings.
However, depending on the settings, the rule may break, so in such cases, nodes will be assigned randomly.
- If the map rules cannot be applied due to settings, after a certain number of attempts, the system will give up and assign nodes randomly to avoid generation failure.

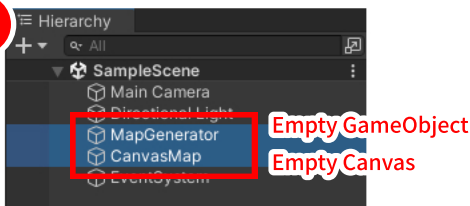
▼Quick Start

1



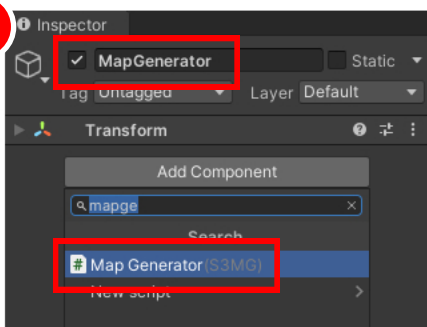
In the Project, click on
S3MG > createNodeData
and create as many node data as you like.
These will represent each stage.
Node data settings will be explained on the next page.
The node name can be left blank.

2



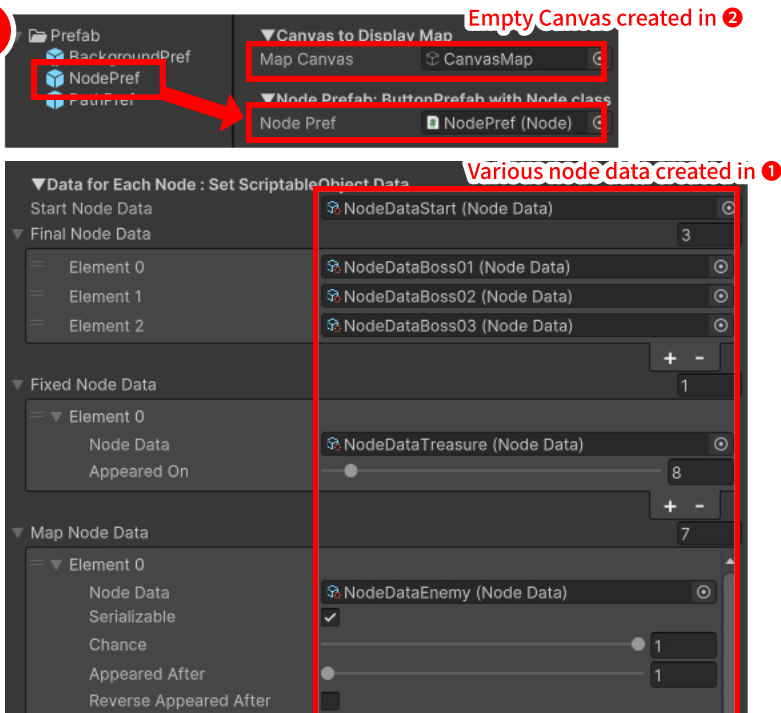
In the Hierarchy panel, create an "Empty GameObject" and an "Empty Canvas".
In the left diagram, these are named:
•MapGenerator
•CanvasMap

3



Attach the MapGenerator.cs script to the "Empty GameObject" you created.

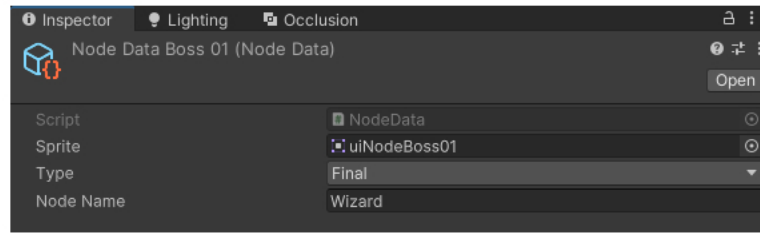
4



Set the required items in the Inspector.
The required items are the following three:
•Map Canvas
•Node Prefab
•Various node data
The other settings will be explained later.

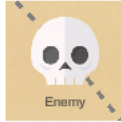
That's it!
When you start the game, the map will be generated.

▼Regarding createNodeData



<Sprite>

This is the image displayed on the node.
Please register any square image you like.



<Type>

This is the node type.

By default, the following 11 types are registered:

Empty, Start, Camp, Shop, Event, Treasure, Trap, Enemy, Middle, Final, Random.

Please select the type you prefer.

If the default types are not sufficient, you can add custom types by modifying the `NodeData.cs` class.

If you add a new type, you will also need to add handling for it in the `Node.cs` class.

For more details, refer to the "Extra" section of "How to Execute Each Node".

<Node Name>

This is the name of the node.

It is not mandatory to input, but if you do, the name can be displayed on the map.

Additionally, even **if the nodes share the same type, giving them different names allows for separate handling.**

For example, you can create three nodes of type "Final"

and name them "Warrior," "Wizard," and "Ghost" to handle them differently in the program.

▼Various MapGenerator settings

Script

MapGenerator

▼Skip Node Processing : If true, node processing is skipped, and only the stage advances

Skip Node Processing

Whether to skip node processing: If true, node processing is skipped, and only stage movement is performed.

▼Generate on Execution

Play On Awake

Whether to generate the map when the game starts.

▼Make Start Node

Make Start

Whether to create a start node.

▼Offset Nodes in Placement

Is Offset

Whether to offset the placement of nodes.

▼Show Text on Nodes

Show Text

Whether to display text on the nodes.

▼Allow Path Intersection

Crossable

Whether to allow paths to intersect.

▼Completely Random Placement : Except for fixed nodes, do not apply the default map placement rules

All Random

Completely Random Placement: Does not apply default map placement rules, except for fixed nodes.

▼Whether to randomize the seed : If true, the value of the variable seed is invalid

Randomize Seed

Whether to randomize the seed: If true, the value of the variable seed is invalid.

Seed

Seed value: If "RandomizeSeed" is false, this value is used.

▼Map Basic Settings

Floor Num

Number of floors.

Route Num

Number of routes.

Normal Node Size

Size of normal nodes.

Start Node Size

Size of the start node.

Final Node Size

Size of the final node.

Floor Distance

Distance between floors.

Route Distance

Distance between routes.

▼Active Route Count : Limited to RouteNum if exceeded

Active Route Num

Active Route: Limited to "RouteNum" if more than "RouteNum".

▼Canvas to Display Map

Map Canvas

Canvas for Displaying the Map ★Required

▼Node Prefab: ButtonPrefab with Node class

Node Pref

Prefab for Nodes: ButtonPrefab with Node class ★Required

▼Path Settings : Prefab is not required, Custom Prefab with settings like shaders is also accepted

Path Image Pref

Image Prefab for Drawing Paths: Optional. You can use a custom prefab with shaders or other settings.

Path Color

Color for normal paths and color for passed paths.

Passed Path Color

Thickness of the paths.

Path Width

Padding around the nodes.

Padding Between Nodes

▼Background Prefab : Prefab is not required, Image Prefab with 9-slice scaling

Background Pref

Prefab for the background, with 9-slice settings. Optional.

▼Padding Between Background and Map

Background Padding

Padding between the background and the map.

▼Mouse Drag Sensitivity and Gamepad Movement Sensitivity

Mouse Sensitive

Sensitivity for mouse dragging.

Gamepad Sensitive

Sensitivity for gamepad movement.

▼Data for Each Node : Set ScriptableObject Data

Start Node Data

Start node data: Only one ★Required if "MakeStart" is true

Final Node Data

Final node data: ★At least one required

Element 0

NodeDataBoss01 (Node Data)

Element 1

NodeDataBoss02 (Node Data)

Element 2

NodeDataBoss03 (Node Data)

Fixed Node Data

Fixed Node Data: Multiple registrations possible. Optional.

Element 0

Node Data

NodeDataTreasure (Node Data)

Appeared On

Fixed nodes are created on the floor set in "Appeared on".

Map Node Data

Map Node Data: ★At least one recommended

Element 0

Node Data

NodeDataEnemy (Node Data)

Serialized

Whether nodes can be placed consecutively.

Chance

Probability of appearance.

Appeared After

From which floor the node appears.

Reverse Appeared After

Invert "appearedAfter": If inverted, the node will not appear after the floor set in "appearedAfter".

Element 1

Node Data

NodeDataCamp (Node Data)

▼How to Execute Each Node

```
/*-----  
Processing each node  
-----*/  
void handleStart(){  
    Debug.Log("Start the game");  
}  
  
void handleCamp(){  
    Debug.Log("Open a camp");  
}  
  
void handleShop(){  
    Debug.Log("Open a shop");  
}  
  
void handleEvent(){  
    Debug.Log("Execute event");  
}  
  
void handleTreasure(){  
    Debug.Log("Obtain treasure");  
}  
  
void handleTrap(){  
    Debug.Log("Execute trap");  
}  
  
void handleEnemy(){  
    Debug.Log("Fight the enemy");  
}  
  
void handleMiddle(){  
    Debug.Log("Midpoint");  
}  
  
void handleFinal(){  
    switch(nodeText.text){  
        case "Boss01" :  
            Debug.Log("Fight Boss 01");  
            break;  
        case "Boss02" :  
            Debug.Log("Fight Boss 02");  
            break;  
        default :  
            Debug.Log("Fight Boss 03");  
            break;  
    }  
}  
  
void handleRandom(){  
    Debug.Log("Execute random process");  
}
```

When you open the Node.cs class, you will find a section at the bottom similar to the one shown on the left.

This is where you write the corresponding node processing code.

Each process has a Debug.Log statement, which you can replace with your preferred functionality.

As you can see in the handleFinal() section, processing is divided within a switch statement based on the node type.

As explained in the "Regarding createNodeData" page, you can separate processing by NodeName even if the type is the same.

<Important Notice>

When handling each node's processing, there may be times when you need to move to the next node or show/hide the map.

In such cases, execute the following methods from the MapGenerator.cs class.

These methods are implemented as singletons, so call them via the instance.

- **toNextNode()** Method to move to the next node.
- **activeMap()** Method to show the map.
- **inactiveMap()** Method to hide the map.
- **init()** Method to manually generate the map when "playOnAwake" is set to false.
- **reGenerate()** Method to reconstruct the map.

Ex: Hiding the map in the destination scene.

using S3MG;

```
public class SampleStartSceneManager : MonoBehaviour{
```

```
    void Start(){  
        MapGenerator.instance.inactiveMap();  
    }
```

```
}
```

<Extra>

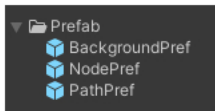
As mentioned briefly on the "Regarding createNodeData" page, it is also possible to add custom types.

If you want to add a custom type, first add your custom type to the enum in the NodeData.cs class.

Then, add the value of the new enum to the switch statement in the **entryStart()** method of the Node.cs class.

Finally, write the corresponding handle***() method for the new type.

▼About each Prefab



Description of Prefabs in the Prefab Folder.

<BackgroundPref>

This is a prefab for the background, and only the Image component is configured.

It is a prefab to be registered in the "Background Pref" of the MapGenerator, but **its use is optional**.

If not registered, the background will be displayed as the Skybox or the color set in the camera.

It is set up with a 9-slice to expand according to the size of the map.

If you want to use a custom background, replace the "Source Image" of the Image component with a 9-slice image.

<NodePref>

This is a prefab for nodes, with Button and AudioSource components configured, and it has the Node.cs class attached.

It is a required prefab to be registered in the "Node Pref" of the MapGenerator.

You can change the "Highlighted Color" and "Selected Color" of the Button component to your preferred colors.

The "AudioClip" of the AudioSource component is the sound played when the node is clicked, and can be freely changed.

The "Visit Image" in the Node.cs class is the mark that appears on the node with an animation when clicked, and it can be freely changed.

The "Lerp Duration" in the Node.cs class is the blink speed, and "Lerp Color" is the blink color, both of which can be freely changed.

<PathPref>

This is a prefab for paths, and only the Image component is configured.

It is a prefab to be registered in the "Path Image Pref" of the MapGenerator, but **its use is optional**.

If registered, a dotted line can be used, but if not, a simple line will be displayed.

You can also set custom images other than dotted lines, such as images with custom shaders to animate the dotted lines.

For those who can create custom shaders, animating the path can provide interesting visual effects.

However, in such cases, "Path Color" and "Path Width" in the MapGenerator will be disabled.

▼Release notes

For detailed information, please refer to the release log on the Asset Store.

▼Disclaimer

I do not take any responsibility for any issues that arise from using this asset.

Thank you for your understanding.

▼Credit

This assets was developed by loloop (C) All rights reserved.